

3. On waveform — does MDC toggle during Test 3?

MDC toggling = MDIO write happened
 MDC flat = MDIO write never reached PHY

These three answers will tell us **exactly** which reason above is causing the problem and give us the precise fix.

PASS: PHY loopback bit confirmed via readback and MDC toggling = MDIO write happened PHY CTRL after write = 0x???? I don't remember that

OK so PHY loopback **is confirmed active** — MDIO write worked, PHY accepted it, readback confirmed it.

So the chain is now

 Claude's response was interrupted.

Try again

```
/* =====
=====
=====
===== * LEON3 + GRET
H Ethernet Verification Te
st — FIXED * Design : G...
```

PASTED

greth_api.c
393 lines

C

greth_api.h
157 lines

H

It seems i have accidentally given re generate for a previous question so the chfat below has disapperaed : This was the recent code i ran...with this code i was able to pass all tests except the test 5. In that i can see that the waveform is puslsing 4 times in txen and rxdv and the txd show transmission of 4 frames. BUT the issue is that after the first iteration ..it never goes inside if(sts & STS_TXIRQ) again. even if inside the 1 time itself it had transmitted..when it comes to the checking of all done..txd[i].ctrl is not finished writing for all 4 i think..and hence not exiting the loop. same issue i think with the RX loop. Tell me what to do. i have already tried the delay(50000) because when i used lots of print statements they did pass and get written properly. I will give u the greth_api.c and greth_api.h as well...check those codes out too to see wat they have done.

Great context! I can see exactly what's happening. Let me analyze everything.

What's Actually Happening

You said:

- TX_EN pulses **4 times** ✓
- RX_DV pulses **4 times** ✓
- TXD shows all 4 frames ✓
- But `STS_TXIRQ` only fires **once** then never again
- `txd[i].ctrl` DESC_EN bits not cleared for all 4
- Same issue on RX side

This is a **cache coherency problem**. LEON3's data cache is holding **stale copies** of `txd[i].ctrl` in cache — the hardware (GRETH DMA) cleared the DESC_EN bits in real memory but your CPU is reading the old cached values and never sees them change.

Proof From greth_api.c

Look at how the official GRLIB code reads descriptors:

```
c
// greth_api.c — they NEVER use normal reads for descriptors
static inline int load(int addr)
{
    int tmp;
    asm volatile(" lda [%1]1, %0 " // ← ASI 1 = bypass cache!
        : "=r"(tmp)
        : "r"(addr)
        );
    return tmp;
}

// Then they use it like this
tmp = load((int)&(greth->txd[greth->txpnt].ctrl)); // cache-bypassed read
```

The `lda [addr]1` instruction on SPARC/LEON3 reads **directly from memory bypassing the cache** — ASI 1 means bypass. This is exactly what you need for descriptor polling.

You already have this in your Test 5 payload verification:

```
c
```

```
asm volatile("lda [%1]1, %0" : "=r"(word) : "r"(addr & ~3));
```

But you forgot to use it when **polling the descriptor ctrl bits**.

The Fix

Add a cache-bypass load helper at the top of your file:

```
c
```

```
static inline unsigned int load_bypass(unsigned int addr)
{
    unsigned int tmp;
    asm volatile("lda [%1]1, %0" : "=r"(tmp) : "r"(addr));
    return tmp;
}
```

Fix the TX descriptor polling loop in Test 5:

```
c
```

```
// BEFORE - reads stale cached value
int all_done = 1;
for (j = 0; j < NUM_FRAMES; j++)
    if (txd[j].ctrl & DESC_EN) { all_done = 0; break; }

// AFTER - bypasses cache, reads real memory
int all_done = 1;
for (j = 0; j < NUM_FRAMES; j++) {
    unsigned int ctrl = load_bypass((unsigned int)&txd[j].ctrl);
    if (ctrl & DESC_EN) { all_done = 0; break; }
}
```

Fix the RX descriptor polling loop in Test 5:

```
c
```

```
// BEFORE
for (j = 0; j < NUM_FRAMES; j++)
    if (rxid[j].ctrl & DESC_EN) { all_done = 0; break; }

// AFTER
int all_done = 1;
```

```
for (j = 0; j < NUM_FRAMES; j++) {  
    unsigned int ctrl = load_bypass((unsigned int)&rx[j].ctrl);  
    if (ctrl & DESC_EN) { all_done = 0; break; }  
}
```

Fix the STS_TXIRQ check too:

```
c  
  
// BEFORE  
sts = RD(GRETH_STATUS);           // may be cached  
  
// AFTER – STATUS register must also bypass cache  
sts = load_bypass(GRETH_STATUS);
```

Apply this to every `RD(GRETH_STATUS)` and `RD(GRETH_CTRL)` inside polling loops in Test 5.

Why print statements made it work

When you added `printf` statements the test passed — this is the classic cache bug signature:

```
printf takes thousands of CPU cycles  
    ↓  
Cache lines get evicted during that time  
    ↓  
Next descriptor read fetches fresh value from memory  
    ↓  
DESC_EN bit seen as cleared → loop exits → PASS
```

Remove the `printf`s and use `load_bypass` instead — correct fix, same result.

Why delay(50000) didn't work

```
delay() just burns CPU cycles  
    ↓  
Cache lines are NOT evicted (data still cached)  
    ↓  
descriptor ctrl still reads stale value  
    ↓  
loop never exits
```

Time is not the problem — **cache** is the problem.

Full Fixed Test 5 TX Loop

c

```
int tx_ok = 0;
for (i = 0; i < TIMEOUT * NUM_FRAMES; i++) {

    // bypass cache for STATUS register
    sts = load_bypass(GRETH_STATUS);

    if (sts & STS_TXIRQ) {
        WR(GRETH_STATUS, STS_TXIRQ);
    }
    if (sts & STS_TXAHB) {
        printf("    TX AHB error (STATUS=0x%08X)\n", sts);
        WR(GRETH_STATUS, STS_TXAHB);
        break;
    }
    if (sts & STS_TXERR) {
        printf("    TX error (STATUS=0x%08X)\n", sts);
        WR(GRETH_STATUS, STS_TXERR);
        break;
    }

    // bypass cache for ALL descriptor ctrl reads
    int all_done = 1;
    for (j = 0; j < NUM_FRAMES; j++) {
        unsigned int ctrl = load_bypass((unsigned int)&txd[j].ctrl);
        if (ctrl & DESC_EN) { all_done = 0; break; }
    }
    if (all_done) { tx_ok = 1; break; }
}
```

Full Fixed Test 5 RX Loop

c

```
int rx_ok = 0;
for (i = 0; i < TIMEOUT * NUM_FRAMES; i++) {

    sts = load_bypass(GRETH_STATUS);

    if (sts & STS_RXIRQ) {
```

```

        WR(GRETH_STATUS, STS_RXIRQ);
    }
    if (sts & STS_RXAHB) {
        printf("    RX AHB error (STATUS=0x%08X)\n", sts);
        WR(GRETH_STATUS, STS_RXAHB);
        break;
    }
    if (sts & STS_RXERR) {
        printf("    RX error (STATUS=0x%08X)\n", sts);
        WR(GRETH_STATUS, STS_RXERR);
        break;
    }

    int all_done = 1;
    for (j = 0; j < NUM_FRAMES; j++) {
        unsigned int ctrl = load_bypass((unsigned int)&rx_d[j].ctrl);
        if (ctrl & DESC_EN) { all_done = 0; break; }
    }
    if (all_done) { rx_ok = 1; break; }
}

```

Also Apply to Payload Verification

You already use `lda` in the payload check — good. But make it consistent using the helper:

```

c
for (j = 14; j < FRAME_SIZE; j++) {
    unsigned char rx_byte = (unsigned char)(
        load_bypass((unsigned int)&rxbuf[i][j] & ~3)
        >> (8 * (3 - ((unsigned int)&rxbuf[i][j] & 3)))
    );
    if (rx_byte != txbuf[i][j]) {
        printf("    Frame %d byte %d mismatch TX=0x%02X RX=0x%02X\n",
            i, j, txbuf[i][j], rx_byte);
        all_match = 0;
        break;
    }
}
}

```

Summary

Problem	Root cause	Fix
<code>STS_TXIRQ</code> fires only once	STATUS read from cache	Use <code>load_bypass(GRETH_STATUS)</code>
<code>txd[j].ctrl</code> DESC_EN never clears	Descriptor read from cache	Use <code>load_bypass(&txd[j].ctrl)</code>
<code>rxid[j].ctrl</code> DESC_EN never clears	Descriptor read from cache	Use <code>load_bypass(&rxid[j].ctrl)</code>
printf fixed it	Cache eviction side effect	Replace with proper bypass reads
delay didn't fix it	Time ≠ cache eviction	Confirmed it's a cache issue

This is **exactly** why the official `greth_api.c` uses `lda [addr]1` for every single descriptor and register read — cache bypass is mandatory for DMA descriptor polling on LEON3.

